



Pimpri Chinchwad Education Trust's
**Pimpri Chinchwad College of Engineering
(PCCoE)**
(An Autonomous Institute)
Affiliated to Savitribai Phule Pune
University(SPPU) ISO 21001:2018 Certified by
TUV



Course: Software Testing and Quality Assurance Laboratory	Code: BIT26PE11
Name: Amar Vaijinath Chavan	PRN: 124B2F001
Assignment 9: Implement and execute REST API tests using the Karate DSL framework, applying behaviourdriven development (BDD) style to validate HTTP endpoints.	

1. Introduction

In modern software applications, RESTful APIs play a critical role in enabling communication between distributed systems. Ensuring that these APIs function correctly is essential for system reliability and performance. API automation testing helps validate API behavior early in the development lifecycle and reduces manual testing effort.

Karate DSL is a modern API automation framework that enables testers to implement and execute REST API tests efficiently using a Behaviour-Driven Development (BDD) approach.

2. What is Karate DSL?

Karate DSL is an open-source test automation framework designed specifically for API testing. It is built on top of Cucumber and uses Gherkin syntax (Given, When, Then) to define test scenarios in a readable and structured format.

Unlike traditional frameworks that require extensive Java coding, Karate allows users to write test cases using a domain-specific language (DSL). It natively supports HTTP requests, JSON and XML processing, assertions, and validations without relying on additional libraries.

3. Importance of Using Karate DSL for API Automation

Karate DSL provides several advantages that make it suitable for modern software development practices:

- **Minimal Coding Requirement**
Tests can be written without creating Java classes, controllers, or helper methods.
- **Native JSON and XML Handling**
Karate supports deep comparison of responses, allowing entire payloads to be validated in a single statement.
- **Readable and Maintainable Tests**
The BDD-style syntax makes test cases easy to understand for developers, testers, and business stakeholders.
- **Fast Setup and Execution**
Karate requires minimal configuration and can be executed easily using Maven or an IDE.

4. Traditional API Testing vs. Karate DSL

Feature	Traditional API Testing (RestAssured/JUnit)	Karate DSL
Code Complexity	High – requires Java classes and methods	Low – Gherkin-based scripts
Assertion Style	Manual, field-by-field	Deep and fuzzy matching
Data Parsing	External libraries required	Built-in JSON/XML support
Execution Setup	Time-consuming	Simple and fast

5. Behaviour-Driven Development (BDD) with Karate DSL

Behaviour-Driven Development (BDD) focuses on defining application behavior using clear, business-readable language. Karate DSL fully supports BDD by structuring test scenarios into:

- **Given** – Defines the preconditions and API configuration
- **When** – Executes the HTTP request
- **Then** – Validates the response status and payload

This approach ensures that API tests also act as living documentation of system behavior.

6. Advanced Features of Karate DSL

6.1 Fuzzy Matching

Karate supports fuzzy matching to validate response data types instead of exact values, making tests more resilient to dynamic data.

6.2 Data-Driven Testing

Using Scenario Outline, Karate allows the same test to be executed with multiple datasets sourced from CSV or JSON files.

6.3 Reusable Authentication (Call Single)

Authentication logic can be executed once and reused across all test scenarios.

Java Interoperability

Karate can directly invoke Java methods for advanced processing such as encryption or unique data generation.

6.5 API Mocking

Karate provides built-in support for mocking APIs, enabling testing even when dependent services are unavailable.

7. Execution and Reporting

Karate automatically generates an interactive HTML report after test execution.

Report Features:

- Step-by-step execution results

- Detailed request and response logs
- Execution time for each API call
- No additional configuration required

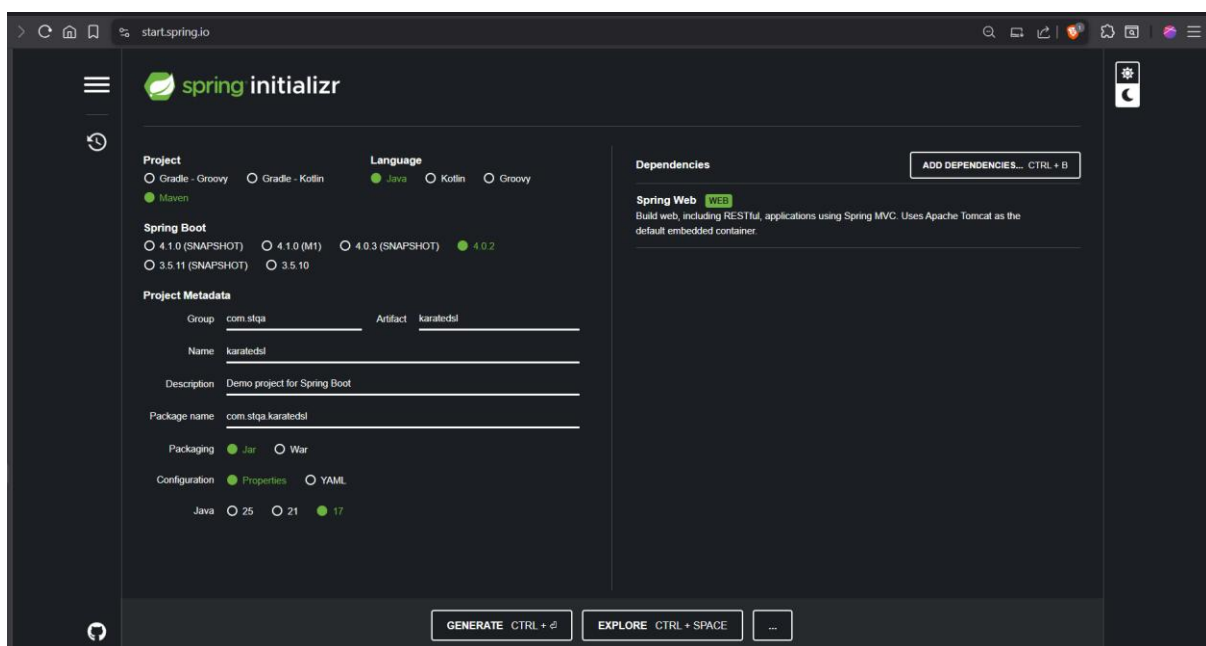
Reports are generated in the target/karate-reports/ directory.

8. Implementation Steps

To perform REST API testing using the Karate DSL framework, it is first necessary to develop the APIs that will be tested. A Java-based Spring Boot project is used for this purpose.

Step 1: Create a Java-Based Spring Boot Project

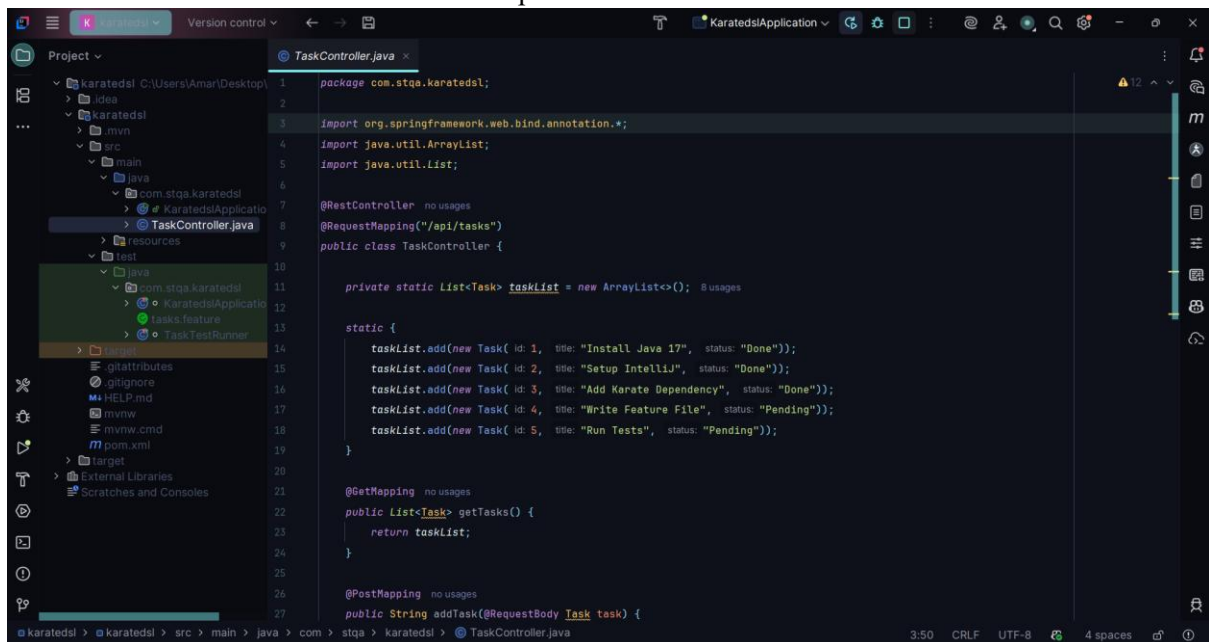
- Open the **Spring Initializr** website: <https://start.spring.io>
- Select **Project**: Maven
- Select **Language**: Java
- Select **Spring Boot version** (default)
- Enter **Group** and **Artifact** name
- Add dependencies: **Spring Web**
- Click **Generate** and extract the project
- Open the project in an IDE (IntelliJ / Eclipse)



Step 2: Develop REST API Endpoints

- Navigate to: src/main/java
- Create a **Controller class**
- Annotate the class with `@RestController`
- Define REST endpoints using annotations such as:
 - `@GetMapping`
 - `@PostMapping`
 - `@PutMapping`

- @DeleteMapping
- Write the code and save the API implementation code



Code to add in the TaskController.java

```

package com.stqa.karatedsl;

import org.springframework.web.bind.annotation.*;
import java.util.ArrayList;
import java.util.List;

@RestController
@RequestMapping("/api/tasks")
public class TaskController {

    private static List<Task> taskList = new ArrayList<>();

    static {
        taskList.add(new Task(1, "Install Java 17", "Done"));
        taskList.add(new Task(2, "Setup IntelliJ", "Done"));
        taskList.add(new Task(3, "Add Karate Dependency", "Done"));
        taskList.add(new Task(4, "Write Feature File", "Pending"));
        taskList.add(new Task(5, "Run Tests", "Pending"));
    }

    @GetMapping
    public List<Task> getTasks() {
        return taskList;
    }

    @PostMapping
    public String addTask(@RequestBody Task task) {
        taskList.add(task);
        return "Task added successfully!";
    }
}

```

```

    @DeleteMapping("/{id}")
    public String deleteTask(@PathVariable int id) {
        boolean removed = taskList.removeIf(t -> t.getId() == id);
        return removed ? "Task deleted!" : "Task not found";
    }
}

class Task {
    private int id;
    private String title;
    private String status;

    public Task() {}
    public Task(int id, String title, String status) {
        this.id = id;
        this.title = title;
        this.status = status;
    }

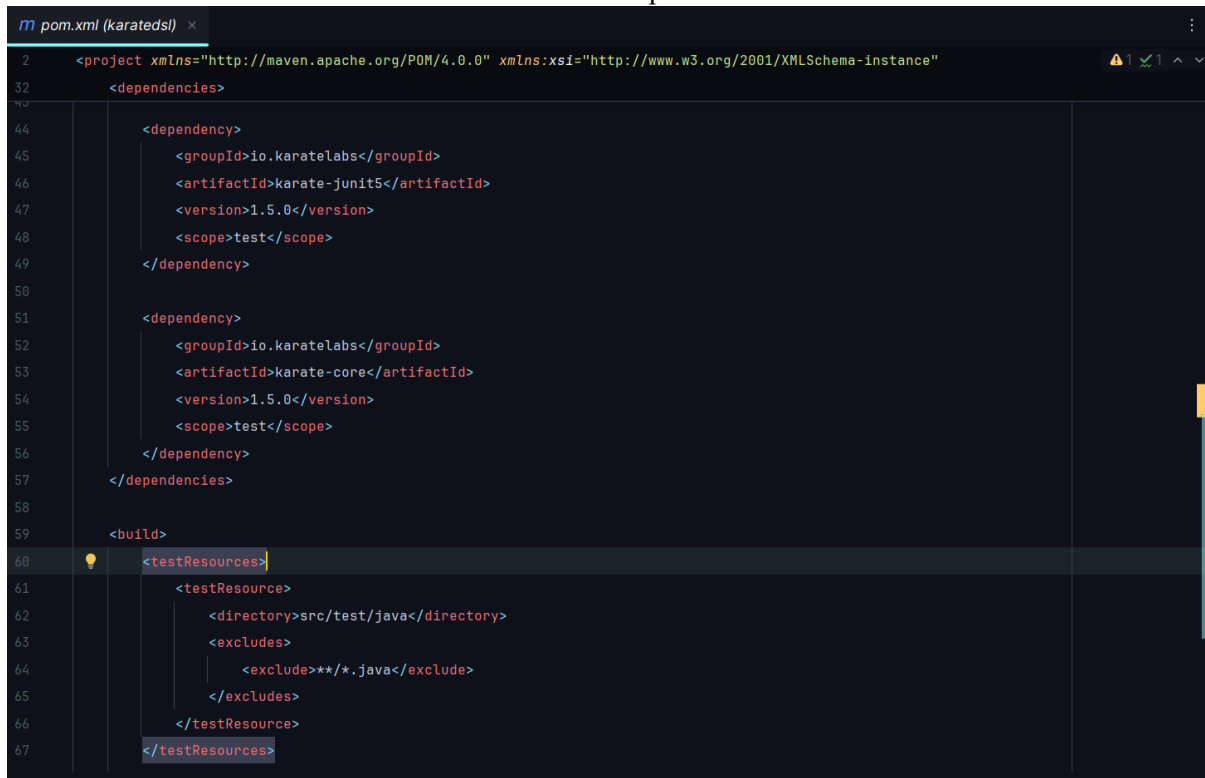
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }
    public String getStatus() { return status; }
    public void setStatus(String status) { this.status = status; }
}

```

Note:- check the package according to your project name it may cause error if mismatch

Step 3: Configure Maven with Karate Dependency

- Open the pom.xml file
- Add the **karate-junit5** dependency inside the <dependencies> section
- Save the file and allow Maven to download dependencies

A screenshot of an IDE window titled 'pom.xml (karatedsl)'. The XML content is as follows:

```
2 <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
32 <dependencies>
44     <dependency>
45         <groupId>io.karatelabs</groupId>
46         <artifactId>karate-junit5</artifactId>
47         <version>1.5.0</version>
48         <scope>test</scope>
49     </dependency>
50
51     <dependency>
52         <groupId>io.karatelabs</groupId>
53         <artifactId>karate-core</artifactId>
54         <version>1.5.0</version>
55         <scope>test</scope>
56     </dependency>
57 </dependencies>
58
59 <build>
60 <testResources>
61     <testResource>
62         <directory>src/test/java</directory>
63         <excludes>
64             <exclude>**/*.java</exclude>
65         </excludes>
66     </testResource>
67 </testResources>
```

Code to add in the pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-
4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>4.0.2</version>
    <relativePath/> <!-- lookup parent from repository -->
  </parent>
  <groupId>com.stqa</groupId>
  <artifactId>karatedsl</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>karatedsl</name>
  <description>Demo project for Spring Boot</description>
  <url/>
  <licenses>
    <license/>
  </licenses>
  <developers>
    <developer/>
  </developers>
```

```

<scm>
  <connection/>
  <developerConnection/>
  <tag/>
  <url/>
</scm>
<properties>
  <java.version>17</java.version>
</properties>
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webmvc</artifactId>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webmvc-test</artifactId>
    <scope>test</scope>
  </dependency>

  <dependency>
    <groupId>io.karatelabs</groupId>
    <artifactId>karate-junit5</artifactId>
    <version>1.5.0</version>
    <scope>test</scope>
  </dependency>

  <dependency>
    <groupId>io.karatelabs</groupId>
    <artifactId>karate-core</artifactId>
    <version>1.5.0</version>
    <scope>test</scope>
  </dependency>
</dependencies>

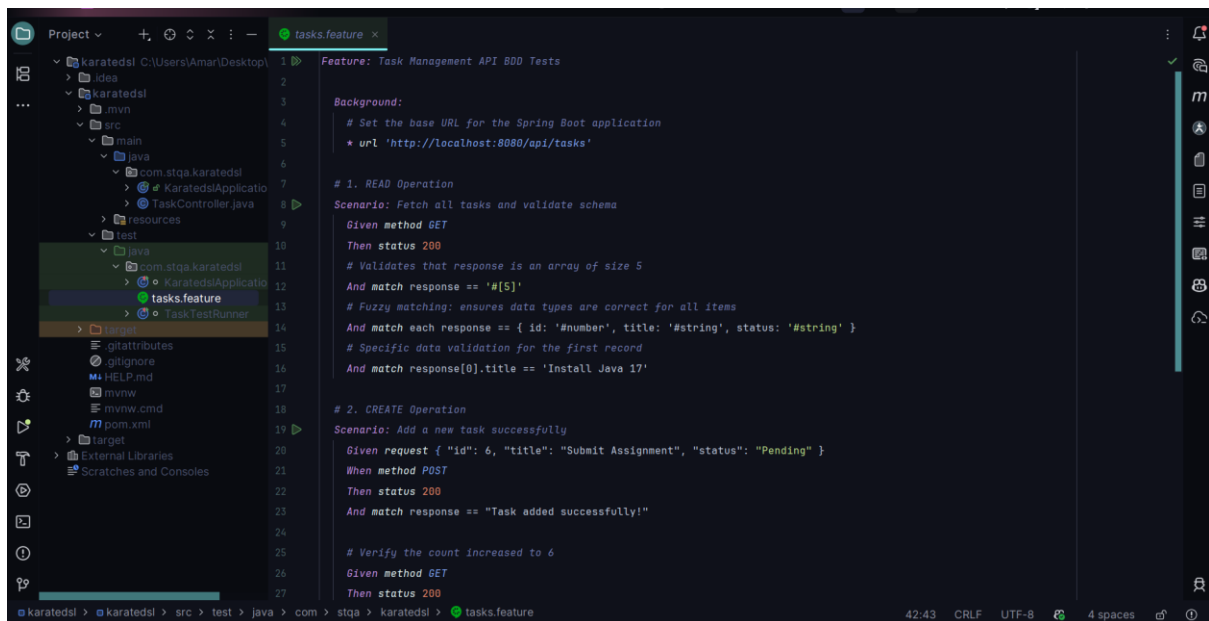
<build>
  <testResources>
    <testResource>
      <directory>src/test/java</directory>
      <excludes>
        <exclude>**/*.java</exclude>
      </excludes>
    </testResource>
  </testResources>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>

</project>

```

Step 4: Create Karate Feature Files

- Navigate to: src/test/java
- Create a package for Karate tests
- Create a .feature file
- Write test scenarios using **Gherkin syntax**:
 - Given – API URL and headers
 - When – HTTP request method
 - Then – Response validation



Code to add in the tasks.feature [test scenarios]

Feature: Task Management API BDD Tests

Background:

```
# Set the base URL for the Spring Boot application
* url 'http://localhost:8080/api/tasks'
```

1. READ Operation

Scenario: Fetch all tasks and validate schema

Given method GET

Then status 200

Validates that response is an array of size 5

And match response == '#[5]'

Fuzzy matching: ensures data types are correct for all items

And match each response == { id: '#number', title: '#string', status: '#string' }

Specific data validation for the first record

And match response[0].title == 'Install Java 17'

2. CREATE Operation

Scenario: Add a new task successfully

Given request { "id": 6, "title": "Submit Assignment", "status": "Pending" }

When method POST

Then status 200

And match response == "Task added successfully!"


```
# Verify the count increased to 6
Given method GET
Then status 200
And match response == '#[6]'
```

3. DELETE Operation (Success Case)

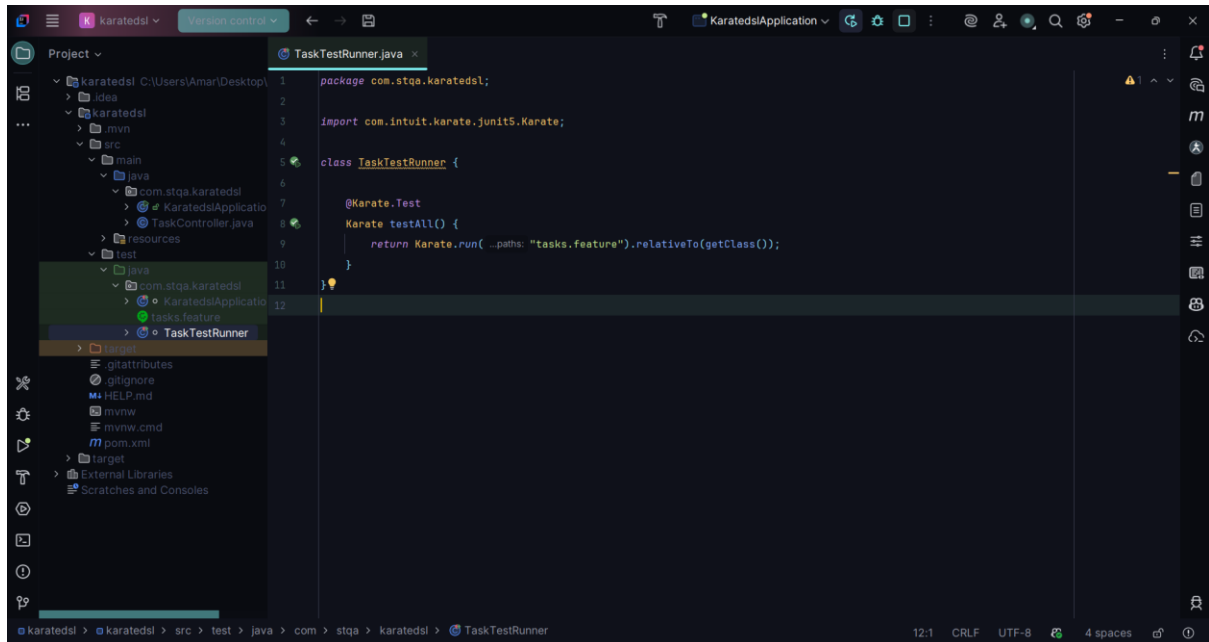
```
Scenario: Delete an existing task
  Given path '1'
  When method DELETE
  Then status 200
  And match response == "Task deleted!"
```

4. DELETE Operation (Failure Case/Negative Test)

```
Scenario: Attempt to delete a task that does not exist
  Given path '99'
  When method DELETE
  Then status 200
  And match response == "Task not found"
```

Step 5: Implement a JUnit 5 Test Runner

- Navigate to: src/test/java
- Create a Java class for the test runner
- Annotate the class with `@Karate.Test`
- Link the runner to the feature files
- Save the runner class



Code to add in the TaskTestRunner.java

```
package com.stqa.karatedsl;

import com.intuit.karate.junit5.Karate;

class TaskTestRunner {
```

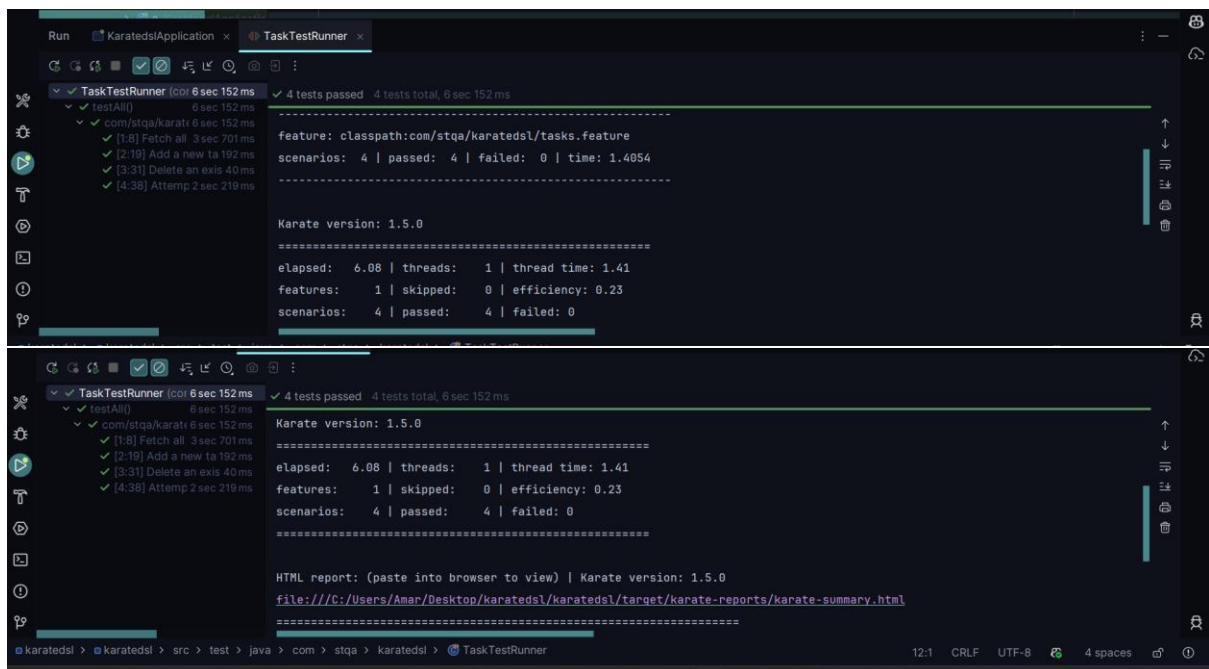
```

@Karate.Test
Karate testAll() {
    return Karate.run("tasks.feature").relativeTo(getClass());
}
}

```

Step 6: Execute the Tests

- Right-click the runner class and select **Run**
OR
- Open terminal and execute:
mvn test



Step 7: Review the HTML Test Report

- Navigate to the target/karate-reports/ directory
- Open the generated **HTML report** in a browser
- Verify test results, request/response details, and execution status

The screenshot shows a web browser displaying the 'Karate Summary Report'. The address bar shows the file path: `C:/Users/Amar/Desktop/karatedsl/karatedsl/target/karate-reports/karate-summary.html`. The page has a dark sidebar on the left with the 'Karate Labs' logo, a 'Features' section showing '1' passed and '0' failed, and a timestamp '2026-02-06 10:00:23 pm'. The main content area has tabs for 'Tags' and 'Timeline'. Below the tabs is a table with the following data:

Feature	Title	Passed	Failed	Scenarios	Time (ms)
com/stqa/karatedsl/tasks.feature	Task Management API BDD Tests	4	0	4	1405

The screenshot shows a detailed view of the Karate HTML Test Report for the feature 'com/stqa/karatedsl/tasks.feature'. The sidebar on the left shows 'Scenarios' with '4' passed and '0' failed, and a timestamp '2026-02-06 10:00:21 pm'. The main content area has tabs for 'Summary', 'Tags', and 'Feature'. The 'Summary' tab is active, showing a list of scenarios and their details. The scenarios are:

- Scenario: [1:8] **Fetch all tasks and validate schema** (ms: 1150)
 - >> Background:
 - # Set the base URL for the Spring Boot application
 - 5 * url 'http://localhost:8080/api/tasks' (143)
 - 9 Given method GET (849)
 - 10 Then status 200 (4)
 - # Validates that response is an array of size 5
 - 12 And match response == '[5]' (95)
 - # Fuzzy matching: ensures data types are correct for all items
 - 14 And match each response == { id: '#number', title: '#string', status: '#string' } (55)
 - # Specific data validation for the first record
 - 16 And match response[0].title == 'Install Java 17' (4)
- Scenario: [2:19] **Add a new task successfully** (ms: 167)
 - >> Background:
 - # Set the base URL for the Spring Boot application
 - 5 * url 'http://localhost:8080/api/tasks' (1)
 - 20 Given request { "id": 6, "title": "Submit Assignment", "status": "Pending" } (1)
 - 21 When method POST (147)
 - 22 Then status 200 (0)
 - 23 And match response == "Task added successfully!" (2)
 - # Verify the count increased to 6
 - 26 Given method GET (12)
 - 27 Then status 200 (0)
 - 28 And match response == '[6]' (4)

com.stqa.karatedsl.tasks	
C:/Users/Amar/Desktop/karatedsl/karatedsl/target/karate-reports/com.stqa.karatedsl.tasks.html	
 4 0	
Scenarios	
2026-02-06 10:00:21 pm	
[1:8] Fetch all tasks and validate schema	
[2:19] Add a new task successfully	
[3:31] Delete an existing task	
[4:38] Attempt to delete a task that does not exist	
# Specific data validation for the first record	
16	And match response[0].title == 'Install Java 17'
	4
Scenario: [2:19] Add a new task successfully	
	ms: 167
>> Background:	
20	Given request { "id": 6, "title": "Submit Assignment", "status": "Pending" }
	1
21	When method POST
	147
22	Then status 200
	0
23	And match response == "Task added successfully!"
	2
# Verify the count increased to 6	
26	Given method GET
	12
27	Then status 200
	0
28	And match response == '#[6]'
	4
Scenario: [3:31] Delete an existing task	
	ms: 25
>> Background:	
32	Given path '1'
	2
33	When method DELETE
	19
34	Then status 200
	0
35	And match response == "Task deleted!"
	3
Scenario: [4:38] Attempt to delete a task that does not exist	
	ms: 64
>> Background:	
39	Given path '99'
	3
40	When method DELETE
	48
41	Then status 200
	0
42	And match response == "Task not found"
	13

9. Conclusion

This assignment successfully demonstrated the implementation and execution of REST API tests using the Karate DSL framework with a Behaviour-Driven Development (BDD) approach. A Spring Boot application was used to develop RESTful endpoints, which were validated using Karate feature files written in Gherkin syntax. Karate DSL simplified API automation by reducing coding effort, improving test readability, and providing powerful validation features. The use of JUnit 5 and automated HTML reports ensured easy execution and clear result analysis. Overall, this approach highlights the effectiveness of Karate DSL for modern, maintainable, and efficient API testing.